

GariGhor Motors — From Brief to Production

A complete, handover-ready car dealership platform with an AI front-desk assistant that captures qualified leads directly into PostgreSQL — built, debugged, and deployed from scratch in a single project sprint.

■ garighor-motors.vercel.app

35+

React components

9

API endpoints

4

DB models

3

Cloud services

2

Lead-capture paths

01 — The Brief

A showroom website that actually works for the owner

The brief was simple but broad: build a production-ready website for a Chattogram-based reconditioned car dealership that could be handed directly to the showroom owner — no developer involvement required for day-to-day use.

That meant solving three distinct problems simultaneously: a public customer site that builds trust and drives enquiries, a private admin console where non-technical staff can manage inventory, and a system that doesn't let leads fall through the cracks at 11 PM when nobody's watching the inbox.

"Most dealership websites are digital brochures. A customer gets curious, doesn't get an answer fast enough, and leaves. I built something that captures them instead."

Customer needs

Browse real inventory, save favourites, submit enquiries and test-drive requests, get instant AI assistance at any hour.

Owner needs

Add/edit/delete listings, upload photos, view all enquiries in one place, never manually transcribe a phone number from a chat log again.

02 — Technical Stack

Every choice justified, every layer understood

No boilerplate generators, no starter kits. Every file was written by hand — including the Prisma schema, all Express middleware, and the entire 1,686-line React application.

Frontend

[React 18](#) · [Vite](#) · [Lucide React](#) · [CSS-in-JS](#)

Single-file React app. No component library — every element styled inline for zero dependency bloat.

Backend

[Node.js](#) · [Express.js](#) · [Prisma ORM](#) · [JWT](#) · [bcrypt](#) · [Multer](#)

RESTful API with JWT-protected admin routes, bcrypt hashing, rate limiting, and async error handling.

Database

[Neon PostgreSQL](#) · [Cloudinary](#)

Serverless Postgres with Prisma migrations. Photos on Cloudinary's free tier — persists through restarts.

AI Layer

[Google Gemini 3.6 Flash](#) · [responseSchema](#) · [Structured Output](#)

responseSchema forces typed JSON on every response — no regex parsing of free text.

Infra

[Vercel](#) · [Render](#) · [GitHub CI/CD](#) · [UptimeRobot](#)

Frontend auto-deploys on push. Backend kept alive via 5-minute pings. Migrations at build time only.

Backups

[pg_dump](#) · [adm-zip](#) · [Windows Task Scheduler](#)

Daily database dump + uploads zip to external drive. Restore tested against a live Neon branch.

03 — The AI Lead-Capture Pipeline

The part that makes this more than a chat widget

Standard AI chat assistants reply to messages. This one is purpose-built to convert conversations into structured, database-ready leads — automatically, at any hour, with no human in the loop.

MSG Customer sends message	API Node.js builds context	AI Gemini typed JSON	CHK Validation contact + dedup	DB PostgreSQL lead written	OUT Dashboard ready to call
--	--	--	--	--	---

The structured output schema

Using `responseSchema` in the Gemini API call forces typed JSON on every message — not just when the model decides to use a tool. The backend always receives a predictable structure it can parse and act on, regardless of what the customer said.

```
const RESPONSE_SCHEMA = {
  type: 'OBJECT',
  properties: {
    replyToUser: { type: 'STRING' }, // shown to customer
    triggerBooking: { type: 'BOOLEAN' }, // true only when all 4 fields collected
    bookingDetails: {
      type: 'OBJECT',
      properties: {
        customerName: { type: 'STRING' },
        contactInfo: { type: 'STRING' },
        appointmentDateTime: { type: 'STRING' },
        vehicleStockNo: { type: 'STRING' },
      }
    },
    financingRequest: { type: 'OBJECT' } // EMI calculator trigger
  },
  required: ['replyToUser', 'triggerBooking']
};
```

Code-enforced validation — not prompt-enforced

The system prompt asks the model to verify contact completeness. But LLM compliance is probabilistic. So the backend enforces it again in code — independently — before anything touches the database.

```
function isValidContact(contactInfo) {
  const trimmed = contactInfo.trim();
  const emailLike = /^[^\s@]+@[^\s@]+\.[^\s@]+$/ .test(trimmed);
  const digits = trimmed.replace(/\D/g, '');
  const phoneLike = digits.length >= 7 && digits.length <= 15;
  return emailLike || phoneLike;
}
```

Duplicate lead prevention A dedup query checks for a lead with the same name, car, and source in the past 10 minutes before writing. Scoped by source so demo testing never blocks real customers.	Auto-retry on 503 Gemini free tier returns 503 'high demand' errors. Backend retries up to 3 times (1s/2s backoff) — verified with a mock fetch test — so spikes are invisible to the customer.
Real failure feedback Each outcome has a specific reply: invalid_contact asks them to re-check, db_error gives a direct phone number. The customer is never told 'booked' when nothing was saved.	EMI calculator If a customer states a down payment, the backend computes an actual monthly estimate using real car price data — appended to the reply without the model inventing a number.

Graceful fallback — the AI is an interface, not a single point of failure

A critical architectural upgrade: Gemini 503 errors used to silently drop leads. The system now has two independent paths to the same outcome — the LLM and a structured form — both posting to the exact same /api/leads Node.js endpoint.



When Gemini is unavailable, the chat widget surfaces a 4-step booking wizard inline — no redirect, no new page. The dealer sees the same structured lead either way. The AI never appears in the data path, only in the user experience.

Auto-retry before fallback Before showing the fallback, the backend retries Gemini up to 3 times (1s/2s backoff). Brief demand spikes recover invisibly — only genuine outages reach the customer.	4-step booking wizard Guaranteed structured capture: Vehicle → Date/Time → Name → Contact → Confirm. Posts to /api/leads directly — same table, same admin dashboard, zero AI dependency.
--	---

04 — Features Built

Everything the showroom actually needs

• Full inventory catalog

Search, filter by brand/body/fuel/price, sort by mileage, year, or price. Brand filter auto-builds from live inventory — no hardcoded dropdown list.

- **Real auth — register & login**

bcrypt-hashed passwords, 30-day JWT sessions, session restored on page load. Admin role gates the dealer console server-side, not just in the UI.

- **Wishlist with auto-reconciliation**

Saved to localStorage, reconciled against live inventory on each load — stale car IDs (deleted listings) pruned automatically.

- **Full inventory CRUD**

Add, edit, delete listings with photo upload to Cloudinary. Status changes (available/reserved/sold) update in one click from the admin table.

- **Leads inbox with pipeline**

All enquiries and AI-captured bookings in one dashboard. Source badge (form vs AI-captured), car, contact, timestamp, status dropdown.

- **Portfolio demo console**

Public sandbox — no credentials needed. Browser-only inventory edits, live AI chat, and real demo leads isolated in the database by source tag.

- **Guaranteed booking wizard**

4-step structured form (Vehicle → Date/Time → Name → Contact) always visible in the demo console. Posts to /api/leads directly — captures a real lead whether Gemini is available or not.

- **Chat widget fallback**

When AI is unavailable, the chat bubble surfaces the booking wizard inline — same widget, no redirect. 'Book directly' toggle always visible so users who prefer a form can switch at any time.

- **Mobile-responsive admin**

Sidebar collapses to a horizontal tab bar on screens under 780px. All CRUD actions work correctly on a phone.

- **Dynamic hero image**

Dealer updates the homepage banner from the admin console. Stored in a singleton Setting row in PostgreSQL — /api/settings GET public, PUT admin.

05 — Problems Found & Fixed

What real-world testing actually surfaces

A portfolio project that never encountered real problems proves nothing. These are bugs that appeared during actual use — each diagnosed from real evidence.

AI

Incomplete contact accepted

Gemini set triggerBooking: true when a customer gave 'prasomiy' as an email. Code-level regex validation added — the model's verdict is double-checked in code before any DB write.

AI

Duplicate leads on confirmation

Customer said 'yes, correct' after booking — model re-emitted triggerBooking: true, creating duplicate rows. 10-minute dedup window added per name + car + source.

Backend

Server crash on async errors

Express 4 doesn't catch thrown errors inside async handlers. A single bad request could take down the process. asyncHandler wrapper added to every route.

Backend

Migration on every cold start

prisma migrate deploy was in the Start Command, adding latency to every wake-up. Moved to Build Command — runs once per deploy, not per request.

Frontend

Wishlist phantom saves

Wishlist IDs became stale after deletions/re-seeds. On load, saved IDs now reconciled against live inventory — pruned automatically, guarded against network failures.

Frontend

Markdown asterisks in chat

Gemini replied with ****bold**** even when told not to. Lightweight renderer added — splits on bold patterns, maps '- ' prefix lines to bullet divs. No library needed.

Infra

Gemini 503 high demand

Render logs showed repeated 503s during testing. Auto-retry with exponential backoff (1s, 2s, max 3 attempts) added — verified with mock fetch test.

Infra

Missing database table

Setting model added to schema.prisma but migration never generated — live route returned 404. Caught by actually hitting the endpoint, not by code inspection.

Architecture

AI as single point of failure

Gemini 503 errors were silently dropping leads — if the AI failed mid-booking, the lead was lost. Fixed by separating LLM (interface) from backend (source of truth): chat widget now falls back to a structured booking wizard posting to /api/leads directly. Two paths, same database.

Architecture

Non-deterministic demo experience

A multi-turn Gemini conversation is probabilistic — showing it to a recruiter depended on the AI holding context across several messages. Fixed with a guaranteed 4-step wizard in the demo console that always works, shown alongside the AI chat as a parallel path.

06 — Production Readiness

The things that don't show up in screenshots

Real backup system

Custom Node.js script: pg_dump to compressed PostgreSQL dump + Cloudinary uploads zipped, auto-rotating to keep the last 14 backups. Restore tested against a live Neon branch.

Uptime monitoring

UptimeRobot pings /api/health every 5 minutes, preventing Render's free tier from sleeping. Email alerts on downtime.

Security done properly

Helmet.js, CORS restricted to frontend origin, bcrypt (cost 10), JWT 30-day expiry, rate limiting on public write endpoints, no API keys in frontend code.

Client handover documentation

A 3-page Word document for a non-technical owner: step-by-step login, adding cars, managing enquiries, backup schedule — plus a technical appendix for any future developer.

07 — The Pitch

What this demonstrates for any business

GariGhor Motors was built as a portfolio piece. But the architecture underneath it — an AI assistant that captures qualified leads directly into a structured database, without a human in the loop — applies to any business where customer conversations currently end without a record.

The technical pitch in plain language

When a customer speaks to the AI on this platform, the model doesn't just chat — it acts as an active data collector. The moment the user expresses intent to book, the code forces Gemini to output a strict JSON schema, locking fields for name, contact, date, and vehicle. The Node.js backend validates that data independently, cross-references it against live inventory, prevents duplicates, and writes a structured lead row to PostgreSQL. The dealer opens their dashboard the next morning to a ready-to-call lead, correctly attributed, with no manual work anywhere in the chain.

I can build this exact automated pipeline for any business with customer conversations that currently end without a record: clinics, real estate agencies, service businesses — anywhere a lead falling through at 9 PM costs money.